Static embeddings are back

Model2Vec (Tulkens & van Dongen, 2024) and static retrieval models (Aarsen, HF blog, Jan 2025)

Timeline talk · paper #6

Martin Leiva Vladimir Herrera Nativi

Friday 13 November 2026

IR 2 · Université Paris-Saclay

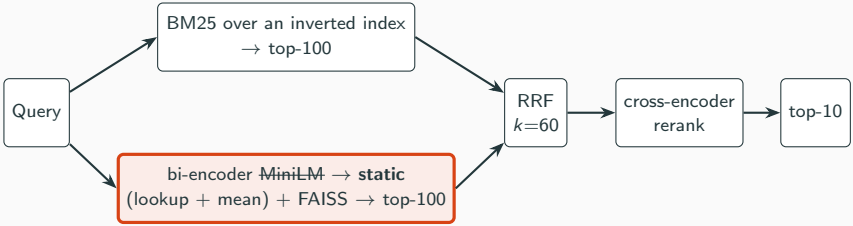
The problem: dense retrieval costs a Transformer pass per text

- A bi-encoder runs every query (and every document) through 6–12 attention layers
- On our laptop CPU: **all-mpnet-base-v2 encodes 104 queries/s**, all-MiniLM-L6-v2 663
- BM25 needs no model, but misses paraphrases and synonyms (vocabulary mismatch)
- Where this hurts: CPU-only servers, phones and browsers, re-indexing millions of documents

Question. Can a dense retriever run at the cost of a dictionary lookup, and what does it lose?

The claim, in one sentence. Static embedding models (token lookup + mean), trained contrastively with a Matryoshka loss, are **100–400× faster on CPU** than common Transformer embedders and keep **≥85 %** of their retrieval quality (NanoBEIR).

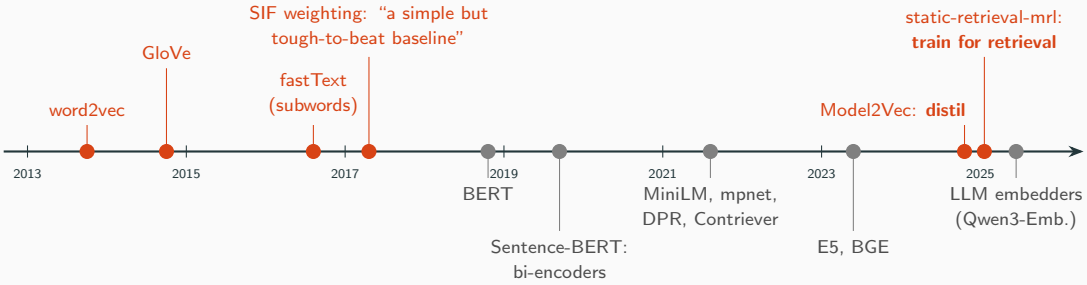
The spring pipeline vs what this paper changes



- **Spring:** BM25 + MiniLM bi-encoder → RRF → cross-encoder, scored with nDCG@10 / Recall@100
- **This paper:** keep everything, replace the bi-encoder's Transformer by a 30k × 1024 table. Index, FAISS, fusion and reranking are unchanged

2013 → 2025: the architecture never changed, the training signal did

static: one fixed vector per token



contextual: one vector per token in its context (attention) (surveys: Kashyap et al. 2024; Zhang et al. 2025)

era	how the table E is filled	NanoBEIR
2013–14 word2vec, GloVe	co-occurrence in raw text	0.329
2024 Model2Vec	*distilled* from a sentence Transformer, no data	0.385–0.443
2025 static-retrieval-mrl	*trained on the retrieval task* (contrastive)	0.503

The method: one figure

(a) Encoding = look up + average

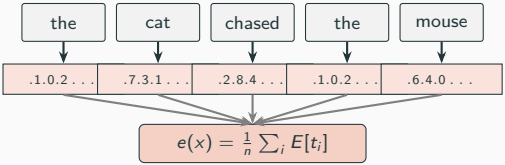


table E: 30,522 tokens $\times$ 1024 dims
no attention, no positions
"the mouse chased the cat"
$\rightarrow$ the *same* vector (cos = 1.000)

(b) Two ways to fill the table E

Model2Vec (2024): distil, no data
1. each token alone through a Transformer: [CLS] t [SEP]
2. PCA $\rightarrow$ 256 dims
3. Zipf/SIF weight $w = \frac{a}{a+p(t)}$, $p \propto \frac{1}{\text{rank}}$

Aarsen (2025): train for retrieval
random E, (query, passage, hard neg.) triplets
MNRL: $-\log \frac{e^{s \cos(q, d^+)}}{\sum_j e^{s \cos(q, d_j)}}$, batch 2048
+ Matryoshka: same loss on $e_{[:32]}, \ldots, e_{[:1024]}$

The method: why each piece is there

Model2Vec (distillation)

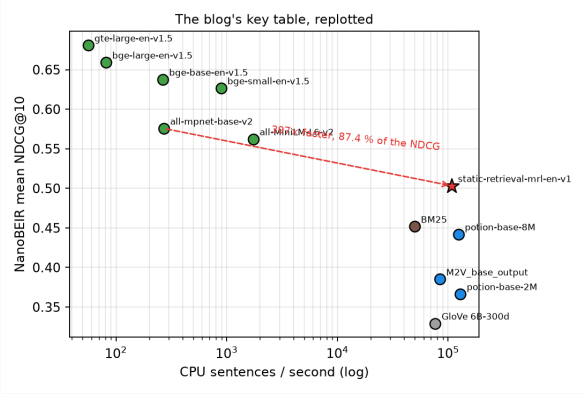
- The Transformer sees each token *alone*: its output is a context-free token vector
- **PCA** centres the space and orders axes by variance
- **Zipf/SIF**: rank $\approx$ frequency, so "the" gets $w \approx 0.002$, rare tokens $w \approx 0.97$. It is a static IDF (Arora et al. 2017)

Aarsen (contrastive training)

- **MNRL / InfoNCE** with in-batch negatives: the spring class-3 loss, applied to a lookup table
- 13 retrieval datasets, batch 2048, **lr 0.2** ($10^4\times$ the usual), 1 epoch, 17.8 h on one RTX 3090
- **Matryoshka**: the same loss on the first 32, 64, . . . , 1024 dims, so a prefix is a usable embedding

Both produce the *same object*: `EmbeddingBag(30522, d, mode="mean")`. Inference = tokenise + read n rows + average.

Their evidence: the one table that matters



Replotted by us from the blog's NanoBEIR table (Aarsen 2025; i7-13700K CPU). Subset of models.

Read for the claim

1. *Claim*: 87.4 % of all-mpnet-base-v2 at 397× its CPU speed; "the only static model to outperform BM25"
2. *Table*: NanoBEIR mean NDCG@10 (13 datasets × 50 queries) vs queries/s
3. *What would falsify it*: a properly configured BM25 above 0.50, which is just as fast and needs no training. BM25 settings are not reported
4. *Measured vs speculated*: speed is measured on *short queries*; "the speed-up only grows with longer texts" is argued, not measured

Reading it critically: what we believe, what we don't

We believe

- The speed numbers: re-run, even larger on our laptop
- The model scores: re-run with an independent evaluator, all within 0.001
- The recipe: we retrain it from scratch and it works

We don't (yet)

- **Weak baseline?** BM25 = 0.452, settings unreported. IR fell for this twice (Armstrong et al. 2009; Lin 2019)
- **Train/test overlap**: MS MARCO is in training and NanoMSMARCO in the test mean. The Transformer baselines were trained on BEIR splits (the blog says so)
- **50 queries per dataset**, one run, no significance test
- **Cost** measured on query encoding only: index size and search time are not discussed

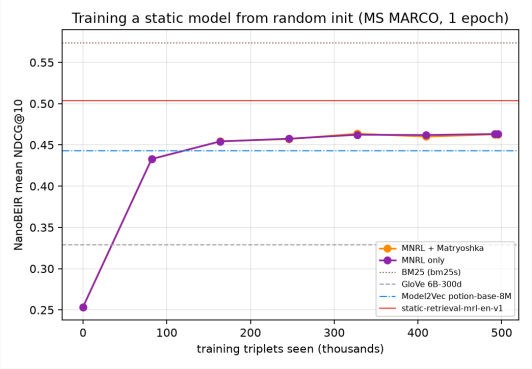
Our number (1): the key table, re-run on our laptop

	NanoBEIR NDCG@10		CPU speed-up vs mpnet	
model	blog	ours	blog	ours
all-mpnet-base-v2	0.576	0.576	1×	1×
all-MiniLM-L6-v2	0.562	0.563	6×	6×
bge-small-en-v1.5	0.627	0.627	3×	4×
static-retrieval-mrl-en-v1	0.503	0.504	397×	543×
potion-base-8M (Model2Vec)	0.442	0.443	459×	813×
GloVe 300d	0.329	0.329	283×	665×
BM25	0.452	**0.574**	–	–

- Own evaluator, cross-checked against sentence-transformers' `NanoBEIREvaluator`: 0.5626 = 0.5626
- bge-small matches only *with* its query instruction (undocumented in the blog)
- **BM25: 0.574, not 0.452.** Two independent implementations agree (bm25s 0.574, NumPy 0.566); 17 standard settings give 0.52–0.58; 32 configurations in all (backup). We did not manage to reproduce 0.452

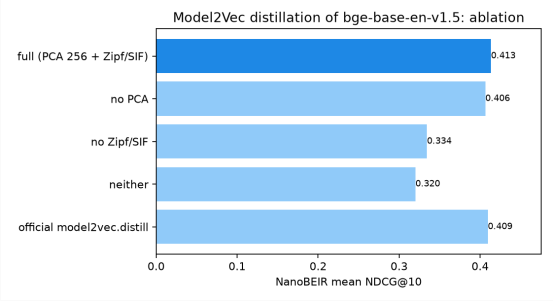
⇒ With a standard BM25, **no static model beats BM25** on NanoBEIR

Our number (2): the recipe on 1 of the 13 datasets



- From scratch, in our own PyTorch: the article's own MS MARCO training set only (493k triplets, 1 of its 13 datasets), blog hyper-parameters, **10 s** of training on an RTX 5050 (+2 min tokenisation)
- **0.4625** $\pm$ **0.0023** (3 seeds) vs 0.503 for the 17.8 h model: above potion-base-8M
- Control: the *untrained* random table already scores **0.25**: averaging random vectors is a random projection of the bag of words, i.e. lexical matching
- Plateau after $\sim$160k triplets: the remaining gap is *data diversity*, not compute
- On the testbed SciFact: 0.580 vs 0.591 (Δ -0.011, $p = 0.34$) , no detectable difference after 10 s vs 17.8 h

Our number (3): Model2Vec from scratch, and what does the work



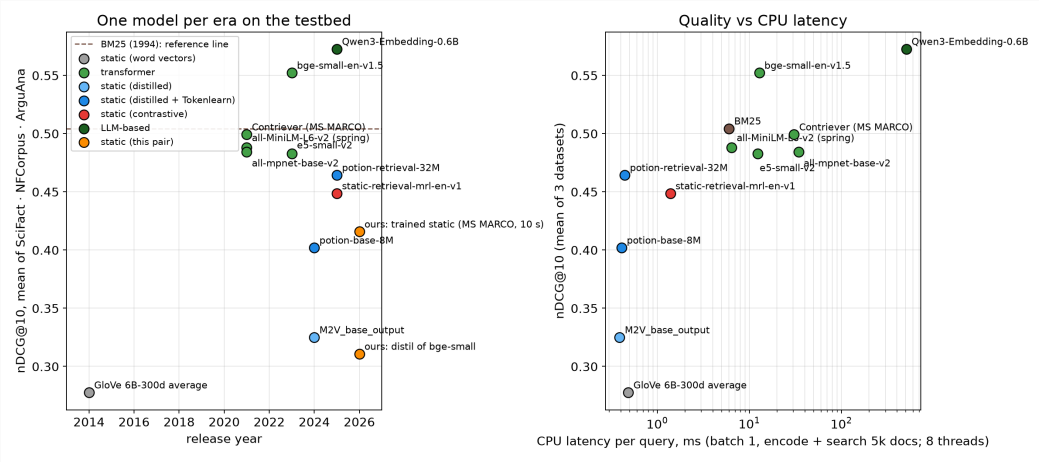
- Our distillation = the official library (max $|\Delta| = 2 \cdot 10^{-6}$); the gap 0.413 vs 0.409 is the library truncating texts at 512 tokens
- **Ablation**: removing Zipf/SIF costs **8** NDCG points, removing PCA 0.7. Mean pooling lets "the", "of", "." dominate; SIF is what makes it a retriever
- Same ordering as Model2Vec's MTEB ablation, but on retrieval Zipf $\gg$ PCA

Our testbed row: static-retrieval-mrl-en-v1 vs the spring MiniLM

dataset	BM25	MiniLM	**static**	Δ vs MiniLM [95 % CI]	p	p_{Holm}
SciFact (300 q)	0.676	0.646	**0.591**	**-0.054 [-0.088, -0.020]**	0.002	0.005
NFCorpus (323 q)	0.321	0.316	**0.301**	-0.015 [-0.034, +0.004] *(noise)*	0.12	0.12
ArguAna (1406 q)	0.516	0.503	**0.453**	**-0.049 [-0.067, -0.032]**	$<$0.001	$<$0.001

- Same `testbed.py`, documents = title + abstract (200 words), paired t-test + bootstrap CI over queries
- On data the authors did not choose: **92 %/95 %/90 %** of MiniLM (SciFact/NFCorpus/ArguAna); significantly *below* MiniLM on 2 of 3 (Holm over the 3 pre-specified tests), and **below BM25 on all 3** (significant on SciFact and ArguAna)
- NFCorpus: -0.015 with $p = 0.12$ is **noise**, not a tie we can claim

One model per era, on the testbed



The whole sweep (colour = Holm-corrected significant vs MiniLM)

system	year	SciFact	NFCorpus	ArguAna	ms/query	dims
BM25	1994	0.676	0.321	0.516	6.0	
GloVe 6B-300d average	2014	0.310	0.137	0.386	0.5	300
all-MiniLM-L6-v2 (spring)	2021	0.646	0.316	0.503	6.4	384
all-mpnet-base-v2	2021	0.648	0.333	0.471	34.2	768
Contriever (MS MARCO)	2021	0.672	0.331	0.494	30.6	768
e5-small-v2	2023	0.680	0.326	0.442	12.3	384
bge-small-en-v1.5	2023	0.710	0.340	0.606	12.8	384
Qwen3-Embedding-0.6B	2025	0.686	0.356	0.675	508.4	1024
M2V_base_output	2024	0.455	0.203	0.317	0.4	256
potion-base-8M	2024	0.523	0.247	0.436	0.4	256
potion-retrieval-32M	2025	0.625	0.312	0.456	0.4	512
static-retrieval-mrl-en-v1	2025	0.591	0.301	0.453	1.4	1024
static-retrieval-mrl-en-v1 @ 384 dims	2025	0.580	0.294	0.446	–	384
ours: distil of bge-small	2026	0.422	0.188	0.323	–	256
ours: trained static (MS MARCO, 10 s)	2026	0.580	0.274	0.393	–	1024
RRF (BM25 + static)	2025	0.673	0.325	0.503	–	

Best static model on the testbed: Model2Vec's **potion-retrieval-32M**, ahead of the paper's model on all three datasets ($\Delta = +0.034, +0.011, +0.003$; not significant after Holm). Qwen3-Embedding-0.6B (2025) is the only model clearly above bge-small, on ArguAna.

What does it cost? (laptop CPU, SciFact)

system	ms/query (CPU, b=1)	encode corpus (5,183 docs)	params	index (MB)	nDCG (mean)
BM25 (rank_bm25)	6.0	–	0	–	0.504
static-retrieval-mrl	1.4	0.7 s	31M	21.2	0.449
potion-retrieval-32M	0.4	0.4 s	32M	10.6	0.464
all-MiniLM-L6-v2	6.4	2 min	23M	8.0	0.488
bge-small-en-v1.5	12.8	5 min	33M	8.0	0.552
all-mpnet-base-v2	34.2	22 min	109M	15.9	0.484
Qwen3-Emb.-0.6B	508.4	8.2 h	596M	21.2	0.573

- Static: 1.4 ms per query, of which only 0.31 ms is encoding: **the flat search over 1024 dims is the bottleneck**
- Index: 1024 dims = 2.7× MiniLM's. Matryoshka to 384 dims costs -0.011 (SciFact, n.s.)
- Qwen3 is the best model on the testbed, but 508 ms per query on CPU and ∼8 h to encode 5k abstracts
- Training: 17.8 h (blog) vs 10 s (ours, 1 dataset). No GPU needed at inference

Where static wins and loses: concrete queries

Wins (first relevant rank: static / MiniLM / BM25)

- *Individuals with low serum vitamin D concentrations have increased risk of multiple sclerosis*: **1** / 16 / 11. The abstract is about *sun exposure*: a learned token association
- *Zoloft* (NFCorpus): **1** / 20 / >100. A brand name tied to depression
- *kohlrabi*, *rickets* (NFCorpus): **1** / >100 / 1. One-word queries: a lookup table is at its best

Losses

- *The locus rs647161 is associated with colorectal carcinoma*: 8 / **1** / **1**. `rs ##64 ##7 ##16 ##1`: an exact identifier dissolved into digit pieces
- *Deletion of $\alpha v \beta 8$ does not result in a spontaneous inflammatory phenotype*: 13 / **1** / **1**. Same story with α `##v ##`β `##8`

SciFact: static beats MiniLM on 49 queries, loses on 98, ties on 153

- Negated claims: Δ = -0.095 vs -0.049; claims with digits -0.079 vs -0.039. Not significant (Mann–Whitney $p = 0.41$, 0.50): a hypothesis, not a finding
- NFCorpus: the longer the query, the worse static does (Spearman ρ = -0.21, $p < 0.001$)
- Per-query nDCG correlates more with BM25 than MiniLM's does (NFCorpus r = 0.86 vs 0.81; ArguAna 0.65 vs 0.53): it fails where BM25 fails, so fusing them adds little

What did not work

- **The BM25 claim**: not reproduced on NanoBEIR, and on the testbed BM25 beats the static model on all three datasets
- **Matryoshka at 512 dims**: our MRL model and the MNRL-only control both lose $\sim$2 %: the difference (0.003) is *within seed noise*. MRL helps clearly only at $\leq$128 dims
- **Our own training plateaus** at 0.46 (NanoBEIR) after 160k triplets: we trained on 1 of the 13 datasets, and on the testbed it is significantly worse than the released model on NFCorpus and ArguAna
- **The hybrid**: RRF(BM25 + static) is no better than BM25 alone; static fails on the same queries as BM25
- **Our error hypothesis** (negation and numbers hurt static) points the right way but is *not significant* ($p = 0.41$, 0.50)
- **Hidden configuration**: bge-small's number needs a query prompt the blog does not mention; GloVe's sentence-transformers module pads to the longest text and ran out of GPU memory at batch 4096
- **Not reproduced at all**: the multilingual similarity model (out of scope for retrieval)

Connections

Spring course

- Same InfoNCE loss as class 3, but the encoder is a table
- The RRF hybrid of class 4 with a static dense half: RRF(BM25 + static) adds nothing over BM25 alone (-0.003, +0.004, -0.013, none significant after Holm) , while RRF(BM25 + MiniLM) adds +0.042 on SciFact
- "Why BM25 is still a baseline" (today's framing): on our testbed it still beats the 2025 static model

Today's other papers

- **LLM2Vec** (10:20): the opposite end of the cost ladder. A 7B decoder made bidirectional vs a 0-layer encoder: how much quality per FLOP?
- **LIMIT** (11:25): a single vector of d dims can only represent so many top-k subsets. Static uses $d = 1024$; Matryoshka truncates it to 64. Does truncation hit that limit first?

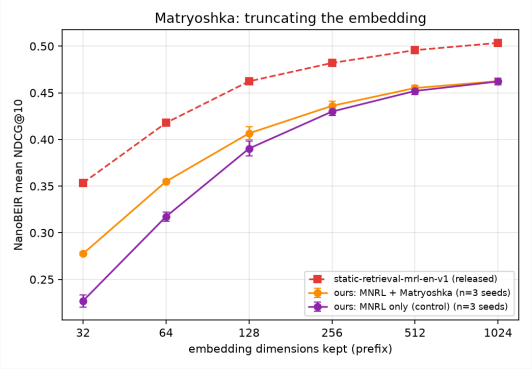
Takeaways and questions for the room

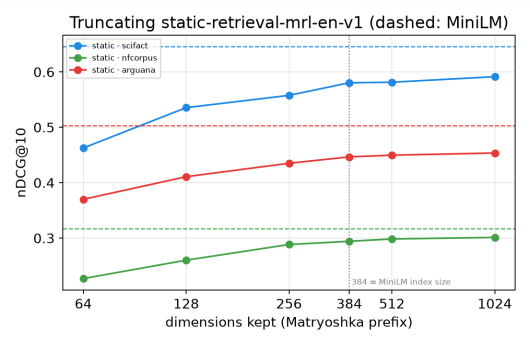
- A static model is a **dictionary**: no context, no word order ("the cat chased the mouse" = "the mouse chased the cat")
- Its speed claim holds and is even stronger on a laptop; its quality claim holds *on the authors' benchmark*
- On the testbed: **90–95 % of MiniLM at 4× lower query latency and 170× faster indexing, but below BM25**, and useless as BM25's hybrid partner. Use it where no Transformer fits (CPU-only, on-device, constant re-indexing), and try Model2Vec's potion-retrieval-32M first: it was the best static model on our data
- The 2013 → 2025 progress is **the training signal**, not the architecture
- **What did the Transformer add?** On the testbed (mean nDCG@10): static 0.449 at 1.4 ms; MiniLM 0.488 at 6.4 ms; bge-small 0.552 at 12.8 ms; Qwen3-0.6B 0.573 at 508 ms ; plus word order, negation and exact identifiers

Discussion. (1) If static vectors get ∼90 % of a Transformer, is contextualisation worth 100–400×? (2) Is progress since 2022 mostly data rather than architecture?

Backup

Backup: Matryoshka on NanoBEIR (3 seeds) and on the testbed





Backup: BM25 tests: can we reach the article's 0.452?

BM25 configuration (NanoBEIR, 13 datasets)	mean NDCG@10
Standard: bm25s lucene, k1 1.5, b 0.75, stopwords + PyStemmer	0.574
5 BM25 variants (Robertson, ATIRE, BM25L, BM25+, Lucene)	0.569–0.574
k1 $\in \{0.9, 1.2, 1.5, 2, 3\}$ × b $\in \{0.4, 0.75, 1\}$	0.523–0.581
Stopword / stemming on or off (both sides)	0.550–0.574
Non-standard: query ids from their own vocabulary	0.003
Non-standard: stemming on one side only	0.389
Non-standard: no lowercasing	0.484
Non-standard: documents cut to 512 / 256 characters	0.527 / 0.478
Non-standard, combined: no lowercasing + 512-character cut	0.433
Article (Aarsen 2025)	**0.452**

No standard setting comes within 0.07 of the article's 0.452: we did not manage to reproduce it. All 32 configurations: `experiments/bm25_diagnosis.py`

Backup: fairness of the comparisons (auditor)

- **Baselines**: `bm25`, `minilm`, `bge` are the testbed's, untouched. Every extra model uses its documented setting: e5 `query:`/`passage:`, Qwen3 query instruction, Contriever dot product, BGE query prefix
- **No tuning on test**: nothing was tuned on the testbed; our trained model never saw the testbed corpora or qrels. Demo queries were *chosen* from test queries for presentation only
- **Multiple comparisons**: Holm correction within two families: the pre-specified official row, and all exploratory variants (3 primary + 66 exploratory tests)
- **Seeds**: training 3 seeds (12, 13, 14), $\pm$ std reported; the bootstrap CI uses the testbed's seed 0
- **Why MiniLM?** It is the box the paper replaces in the spring pipeline. Against `bge`: -0.12 (SciFact), -0.04, -0.15, all $p < 0.001$
- **BM25 on NanoBEIR**: bm25s defaults (k1 1.5, b 0.75) + stopwords + PyStemmer $= 0.574$; NumPy reimplementation 0.566; 32 configurations tried, the blog's setup is still unknown (previous slide)

Backup: reproduction details and deviations

- Code: from scratch (static encoder, Model2Vec distillation, MNRL + Matryoshka, training loop, NanoBEIR evaluator); 19 unit tests, including exact equivalence with `model2vec.distill` and sentence-transformers' losses
- Official code read only to settle ambiguities: `[CLS] t [SEP]` per token, Zipf $p \propto 1/(\mathrm{rank}+2)$, special tokens dropped from the vocabulary, no PCA whitening
- Training: 1 dataset instead of 13 (compute), plain PyTorch loop with the HF Trainer defaults the blog inherits (AdamW, weight decay 0, grad clip 1.0)
- Hardware: RTX 5050 laptop (8 GB), 8 CPU threads; speed compared as *ratios*

References

- T. Aarsen (2025). Train 400x faster Static Embedding Models with Sentence Transformers. Hugging Face blog.
- S. Tulkens, T. van Dongen (2024). Model2Vec: Fast State-of-the-Art Static Embeddings. doi:10.5281/zenodo.17270888.
- T. Mikolov et al. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781.
- J. Pennington, R. Socher, C. Manning (2014). GloVe: Global Vectors for Word Representation. EMNLP.
- P. Bojanowski et al. (2017). Enriching Word Vectors with Subword Information. TACL.
- S. Arora, Y. Liang, T. Ma (2017). A Simple but Tough-to-Beat Baseline for Sentence Embeddings. ICLR.
- N. Reimers, I. Gurevych (2019). Sentence-BERT. EMNLP-IJCNLP. arXiv:1908.10084.
- M. Henderson et al. (2017). Efficient Natural Language Response Suggestion for Smart Reply. arXiv:1705.00652.
- A. van den Oord, Y. Li, O. Vinyals (2018). Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748.
- A. Kusupati et al. (2022). Matryoshka Representation Learning. NeurIPS. arXiv:2205.13147.
- G. Izacard et al. (2022). Unsupervised Dense Information Retrieval with Contrastive Learning. TMLR. arXiv:2112.09118.
- L. Wang et al. (2022). Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv:2212.03533.
- S. Xiao et al. (2023). C-Pack: Packed Resources For General Chinese Embeddings. arXiv:2309.07597.
- Y. Zhang et al. (2025). Qwen3 Embedding. arXiv:2506.05176.
- P. BehnamGhader et al. (2024). LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. COLM. arXiv:2404.05961.
- O. Weller et al. (2025). On the Theoretical Limitations of Embedding-Based Retrieval. arXiv:2508.21038.
- A. R. Kashyap et al. (2024). A Comprehensive Survey of Sentence Representations: From the BERT Epoch to the ChatGPT Era and Beyond. EACL. arXiv:2305.12641.
- M. Zhang et al. (2025). On The Role of Pretrained Language Models in General-Purpose Text Embeddings: A Survey. arXiv:2507.20783.
- N. Thakur et al. (2021). BEIR. NeurIPS Datasets and Benchmarks. arXiv:2104.08663.
- T. Armstrong, A. Moffat, W. Webber, J. Zobel (2009). Improvements that don't add up. CIKM. doi:10.1145/1645953.1646031.
- J. Lin (2019). The Neural Hype and Comparisons Against Weak Baselines. SIGIR Forum. doi:10.1145/3308774.3308781.
- J. Urbano, H. Lima, A. Hanjalic (2019). Statistical Significance Testing in IR: Type I, II and III Errors. SIGIR. arXiv:1905.11096.